Binary Classification of Fake News

Gabriel Vance

DSC 630 – Predictive Analytics

Professor Hua

Introduction of Topic/Problem

Lies have always played a part in communication and reverberated throughout society. Simply put, lies are statements that are not true. Fake news is another term for lies. It presents itself as a legitimate update in the world without maintain a factual basis.

When journalism and technology merged, the dispersion of fake news changed from supermarket tabloids and talk radio to webpages, social media, and now AI generated content. It is increasingly more believable, viral, and widespread. Foreign states, activists, and grifters are spreading fake news to achieve their political, influential and financial ends. But while technology is partially responsible for the rise and proliferation of fake news, it can also substantially aid in combating these lies.

For my project, I chose to use a natural language processor to identify fake news. The assumption was that fake news or lies contained a certain language or pattern structure that stood out from truthful news. This is likely due to fake news largely being “spam” or manufactured news, so in essence, machines were generating fake news and left a “watermark” that was easily predictable, like spam or malware.

For data analysis, we will employ traditional Python data analysis tools in conjunction with NLP libraries such as Text Blob and VADER. For predictive modeling, we will utilize libraries like sklearn for generating classification reports and xgboost for training models.

Business Objective: Train a model that predicts fake news with 75% accuracy.

Overview of Data Used

Collection

The dataset for this notebook was provided by FakeNewsNet's Kaggle dataset [1]. This dataset also contains a robust GitHub repo with a deeper dive into sources as well as API tools. I intentionally chose one of the most popular datasets on Kaggle that has already been downloaded over 10,000 times. The dataset contains 626 real news articles and 434 fake news articles. The news articles themselves have been fact-checked by PolitiFact.

It's important to note that this dataset is crucial to understanding how fake news detectors work. The dataset must be vetted and peer reviewed to avoid having a biased set that will influence the model. One bad data point won't throw off the model, but a strong pattern or trend will corrupt the model. Future models might opt for different datasets to verify fake news can be detected on different sources. These news articles were mostly from the 2016 election year. While this might present itself as a problem, the primary drivers or important features are non-temporal. Future efforts could continue to integrate characteristics of fake news in a non-temporal fashion.

Methods of Analysis

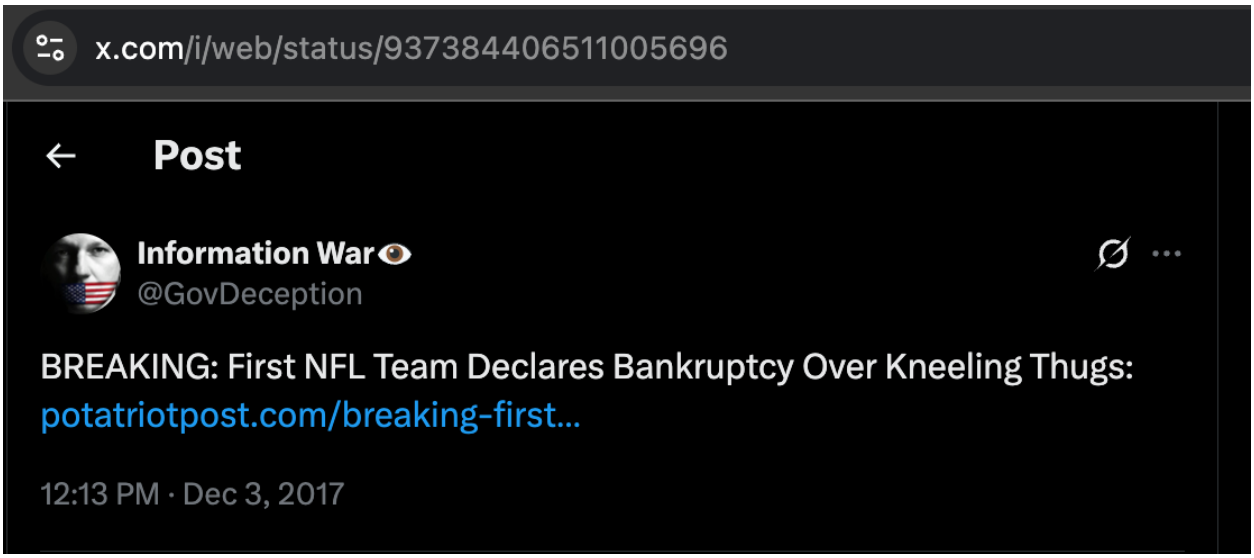
Data Exploration – Initial Summarization

The dataset was comprised of 3 primary features:

1. News_url – news source URL
2. Title – the title of the article
3. Tweet IDs – The tweets related to the article

Each row is a single instance of an article. The title is the headline of the article. The news URL is the source of the article. Tweet IDs are the number of tweets that shared the article. Other fake news datasets contain the actual text of the article, but this isn't required for classifying fake news. Using only the title allows us to expedite model comparison and training time.

Tweet IDs can be used for manually looking up the actual tweet via the twitter API/URL. For example, adding tweet ID “937384406511005696” to the end of URL https://x.com/i/web/status to make https://x.com/i/web/status/937384406511005696 allows us to view the tweet as it appears on twitter.



This is a powerful mechanism as it allows us to tell the story of those spreading fake news. Note the misspelling “potatriotpost” and inflammatory language. We can also inspect the

profile itself to see a very active user with 188k tweets.



Data Exploration – Processing Tweet IDs

Tweet IDs are essential in capturing the resonance of an article. Real and fake news were tweeted at various rates, but the numbers showed a stark contrast in fake news. Before normalization, the mean tweets were as follows:

```
# Calculate mean number of tweet_ids for fake and real dataframes
mean_fake_tweet_ids = politics_fake_df["number_of_tweets"].mean()
mean_real_tweet_ids = politics_real_df["number_of_tweets"].mean()

print(f"Mean # of tweet_ids (fake): {mean_fake_tweet_ids:.2f}")
print(f"Mean # of tweet_ids (real): {mean_real_tweet_ids:.2f}")
```

[26] ✓ 0.0s

```
Mean # of tweet_ids (fake): 382.76
Mean # of tweet_ids (real): 670.13
```

Initial impressions were that fake news was tweeted less than real news. But when controlling for 0s, more than a third of all real news had 0 tweets, whereas only 9% of fake news had 0 tweets. Because many articles were tweeted 0 times and we didn't want to risk data loss, I chose np.log1p to standardize the data. The transformation worked as intended, and the skewness/kurtosis fell close to 0.

```
# Skewness and kurtosis for number_of_tweets
fake_skew_log = skew(politics_fake_df["number_of_tweets_log"])
real_skew_log = skew(politics_real_df["number_of_tweets_log"])
fake_kurt_log = kurtosis(politics_fake_df["number_of_tweets_log"])
real_kurt_log = kurtosis(politics_real_df["number_of_tweets_log"])

print("Before Log Transformation:")

print(f"Fake Articles - Skewness: {fake_skew:.2f}, Kurtosis: {fake_kurt:.2f}")
print(f"Real Articles - Skewness: {real_skew:.2f}, Kurtosis: {real_kurt:.2f}")

print("\nAfter Log Transformation:")

print(f"Fake Articles - Skewness: {fake_skew_log:.2f}, Kurtosis: {fake_kurt_log:.2f}")
print(f"Real Articles - Skewness: {real_skew_log:.2f}, Kurtosis: {real_kurt_log:.2f}")
```

[35] ✓ 0.0s

```
Before Log Transformation:
Fake Articles - Skewness: 13.16, Kurtosis: 196.74
Real Articles - Skewness: 6.55, Kurtosis: 52.10

After Log Transformation:
Fake Articles - Skewness: -0.30, Kurtosis: -0.62
Real Articles - Skewness: 0.61, Kurtosis: -0.95
```

```
    # Calculate mean number of tweet_ids for fake and real dataframes
    mean_fake_tweet_ids = politics_fake_df["number_of_tweets_log"].mean()
    mean_real_tweet_ids = politics_real_df["number_of_tweets_log"].mean()

    print(f"Mean # of tweet_ids (fake): {mean_fake_tweet_ids:.2f}")
    print(f"Mean # of tweet_ids (real): {mean_real_tweet_ids:.2f}")
[34]  ✓  0.0s

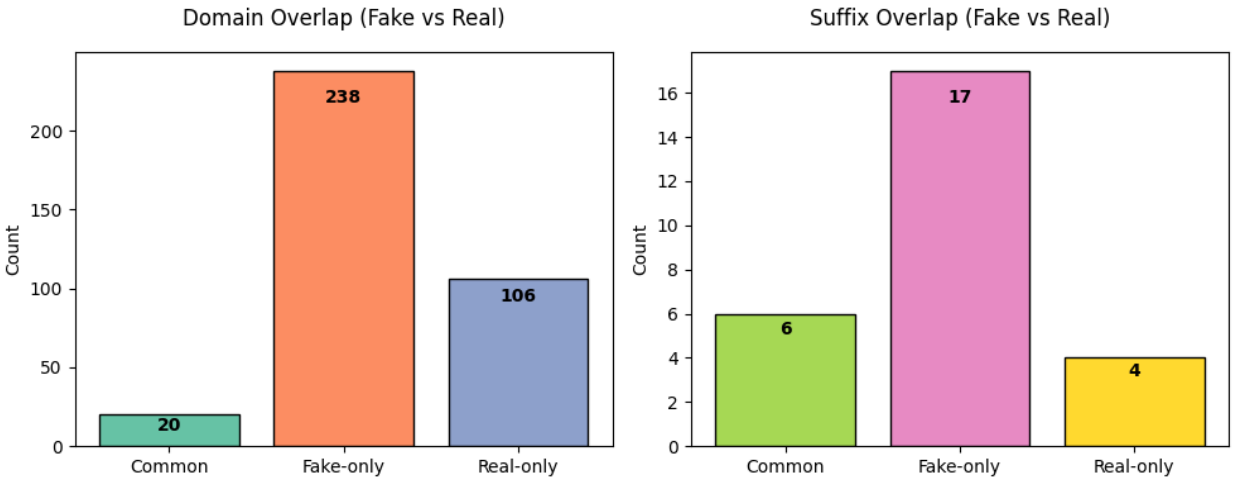
...  Mean # of tweet_ids (fake): 4.04
     Mean # of tweet_ids (real): 2.90
```

Likewise, the number of tweets turned in the opposite behavior: fake news is tweeted 38% more than real news. The original, raw data showed higher tweets for real news due to a handful of anomalies pulling the overall mean significantly higher.

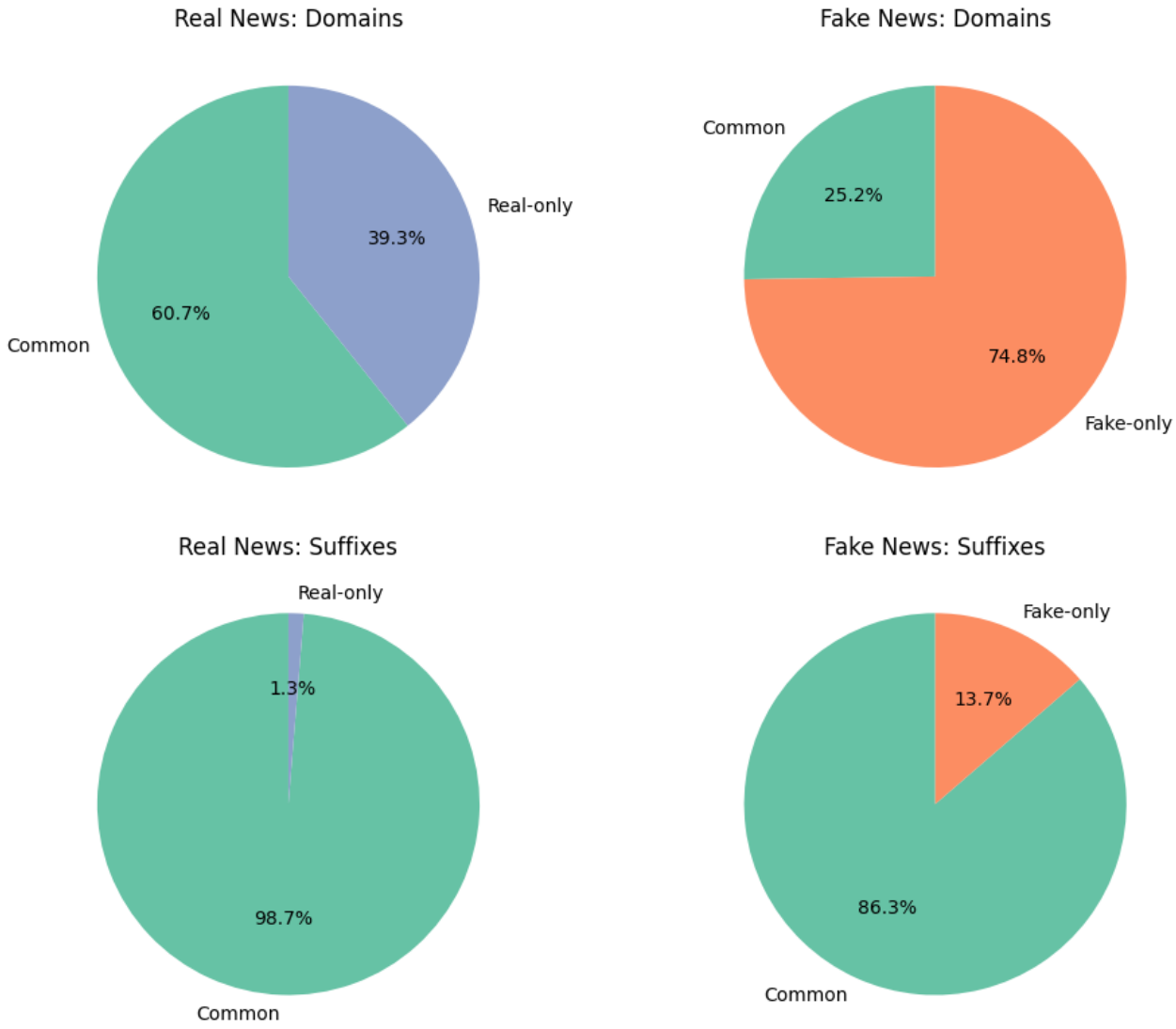
Data Exploration – Processing Domains

Domains are the URLs or address that fake news manifests. Much like antivirus software searches for heuristic signatures within malware, so too can we examine the various characteristics of the domains.

For this data, we created comparisons between fake and real data. Common domains existed in both real and fake data frames while “real-only” and “fake-only” domains, a pattern we would continue throughout the project. Comparing domains vs suffixes, fake-only domains appeared to be a key indicator.



But these numbers only capture unique counts. Pie charts show us what percent of news is dominated by common or real/fake-only domains. Again, we see that fake-only domains make up ~75% of all fake news. This was essential in aiding our predictive efforts.

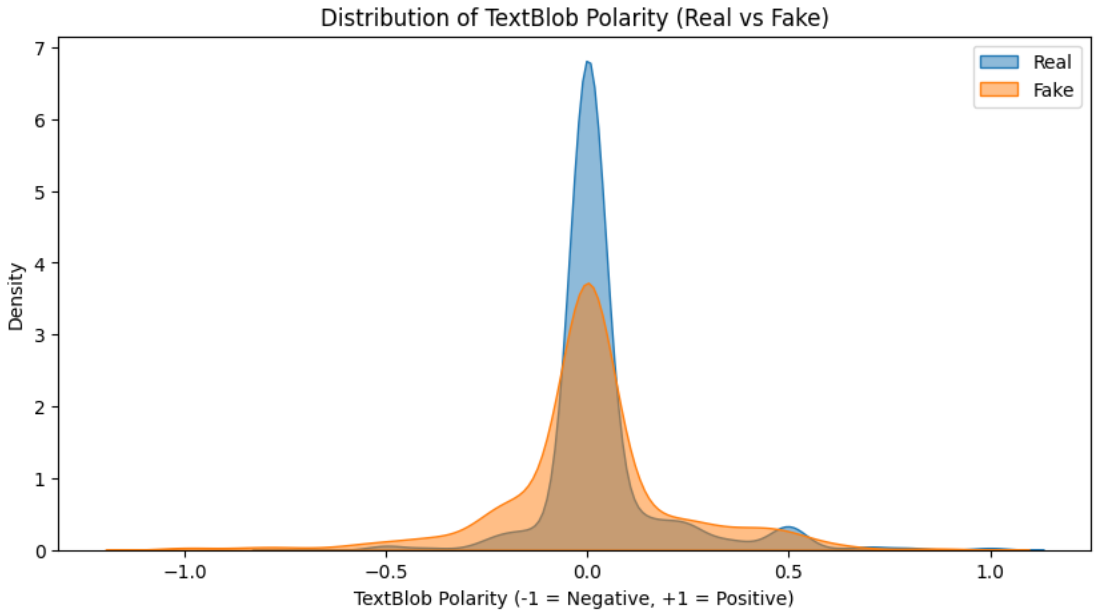


This also provided meaningful data visualizations in the form of word clouds. Word clouds show the frequency of words. When controlling for real-only and fake-only key words, we see various keywords stick out from a visual basis. “whitehouse” and “cbsnews” were high frequency keywords within the real dataset whereas “yournewswise” and “react365” exclusively exist within the fake news.

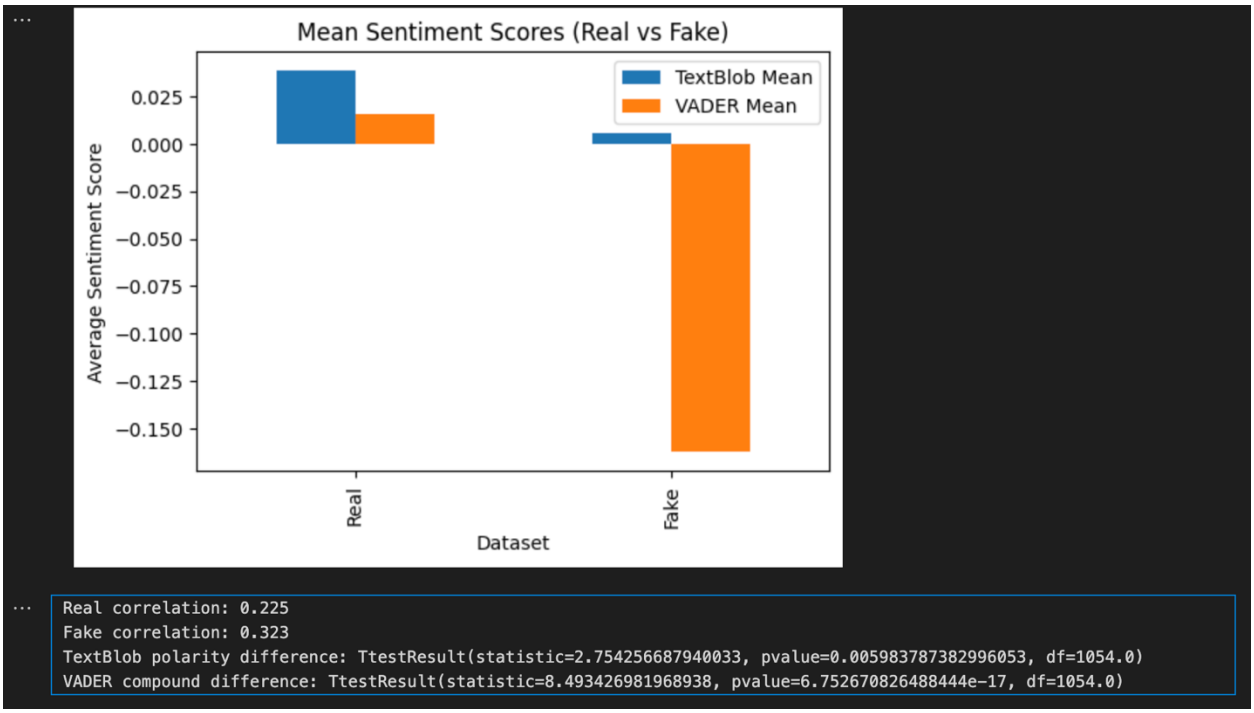


Data Exploration – Processing Titles

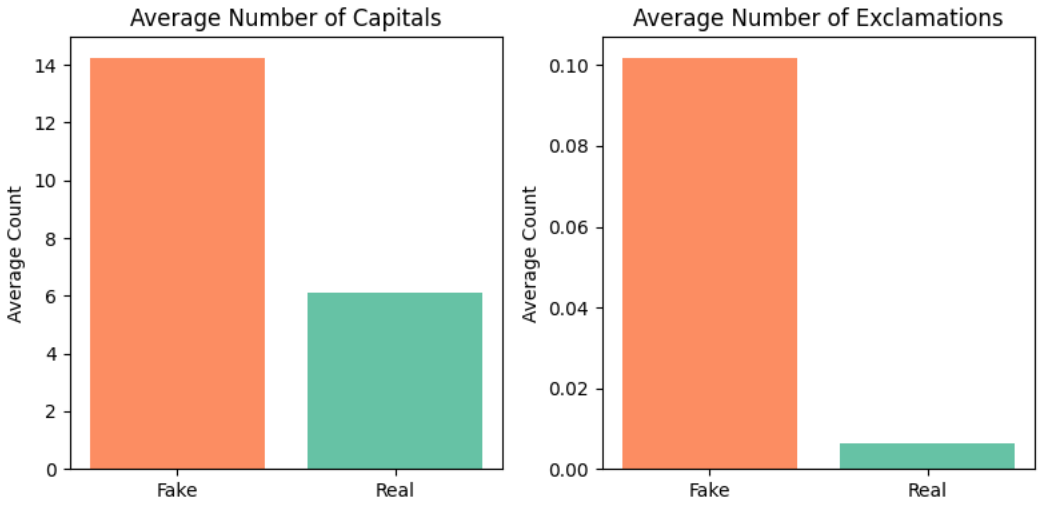
Titles are essential in online journalism and communication. For twitter, titles are the gateway to interest. Inflammatory, polarizing titles drive viewership, or at least our normalized mean tweet counts would seem that way. But now we had to measure just how polarizing and inflammatory these titles were. Utilizing Text Blob and VADER we were able to conduct sentiment values on news. A larger portion of real news maintains a 0.0 polarity, indicating less positive or negative news.



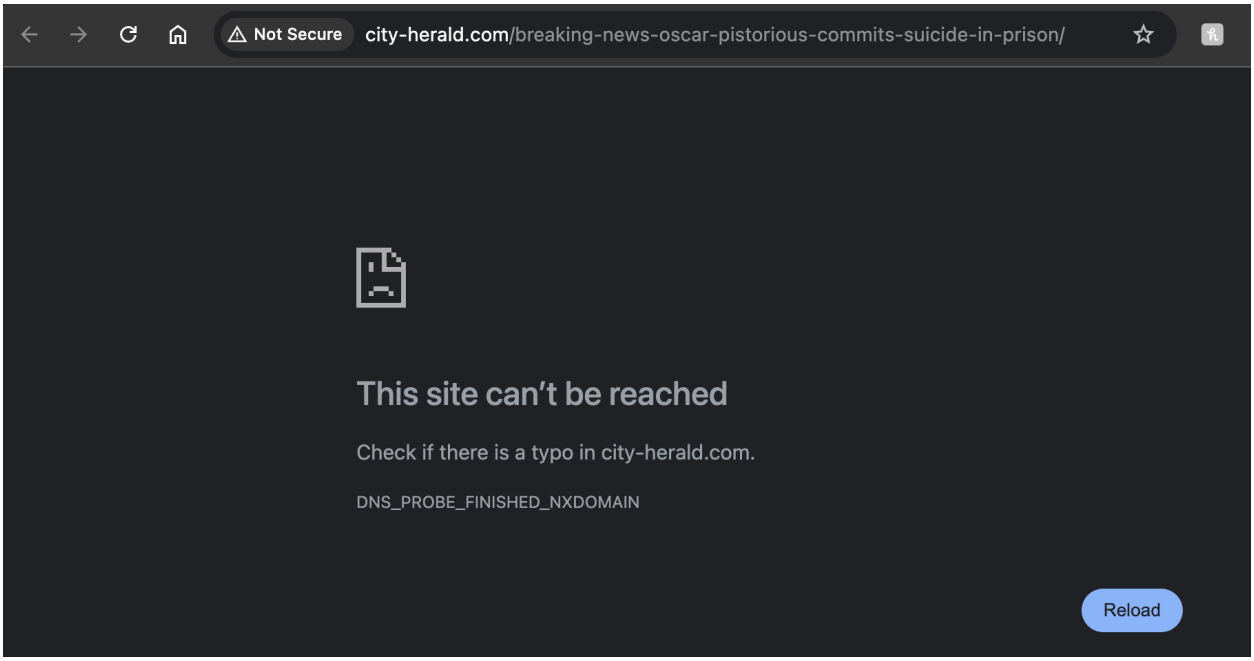
While the sharp contrast is impressive, VADER showed an even larger contrast. So much so that when conducting a t-test, the p-value for VADER sentiment was 6.75e-27 vs Text Blob's 0.006. Both were acceptable for rejecting the null hypothesis, but VADER was more sensitive to the fake news sentiment.



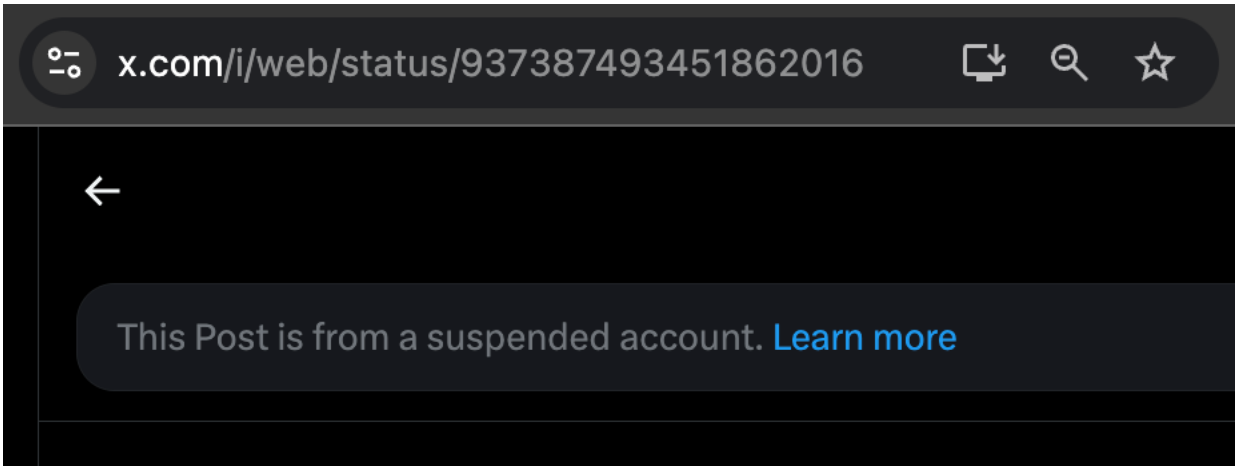
Further emphasizing the use of polarizing language and writing, we introduced two additional features: one for counting the number of uppercase characters and another for the usage of exclamation points. This time, a stark contrast emerges. On average, fake news articles tend to have more capital letters and exclamation marks. For instance, consider our earlier GovDeception tweet titled "BREAKING: First NFL Team Declares Bankruptcy Over Kneeling Thugs". The title itself is formatted in an unusual manner, with each word capitalized, including a full caps "BREAKING".



Returning to manual search, we can paste the articles in our web browser. Unsurprising, a considerable portion of the fake news lives on domains that no longer exist. In other cases, the domain still exists but lacks HTTPS protocol in the address, meaning that the fake news is hosted on an insecure webpage.



Additionally, some accounts that we looked up have since been banned.



In summary, our Exploratory Data Analysis used various tools and methods. For a handful of data points, we manually investigated the data utilizing twitter's API. This was time consuming and not ideal for widespread use, but future work could automate deeper dives into article domains and those who share them. Tweet IDs were heavily skewed and standardizing them revealed the fake news is tweeted more than real news. Checking the kurtosis before and after standardization helped show the effects on the skewed data. Domains were analyzed for the root domain and keywords using tldextract. Domains were categorized based on whether they occurred in fake, real or both datasets. Utilizing pie charts, bar charts, and word clouds, we visualized the disparities and constitution of the data. Text Blob and VADER were utilized for sentiment analysis along with t-testing, indicating that disparities within the data allowed us to reject the null hypothesis. These methods aided us in feature engineering, storytelling, and gave us a deeper understanding of the model.

Results & Findings Explained

Model Evaluation

Model training and evaluation was straight forward. Fake and real data were combined into a single data frame with a “real” label. Fake data had a value of 0 and real data had a value of 1, setting the target for training.

For features, we used the categorical features domain and suffix. Numerical features were Text Blob and VADER sentiment, number of capital letters, number of exclamations, number of words, and number of characters. Title itself was submitted as a Text time, using TfidVectorizer. I fitted a pipeline with a preprocessor that encoded the categorical features.

Using XGBoost, our gradient boost classification model ran with 5 folds and param grid, printing the most optimal parameters alongside feature importance. The small number of estimators (100-120) shows that the simple data structure (along with the relatively small sample size) was able to meet our goal of 75% accuracy for both real and fake news. Recall and F1 also maintained values above 75%, indicating that our model is succeeding in retaining high recall and precision. I was a bit surprised to see suffixes so high in the feature importance as the “common” was concerning.

For good measure, we compared XGBoost, Logistic Regression and Linear SVC. I chose to include these types based on a similar problem by Elsa Norman [3], although Elsa notes that pure accuracy was not the most important indicator because of the influence of quotations within fake news. Because I was processing titles and not full articles, I found the Linear SVC was appropriate when weighing different domains. While each surpassed our goal of 75% accuracy, Linear SVC was the clear winner with 93% accuracy in predicting both real and fake news.

```
=== Classification Report: XGBoost ===
              precision    recall  f1-score   support

   Fake (0)      0.840     0.908     0.873        87
   Real (1)      0.932     0.880     0.905       125

    accuracy                          0.892       212
   macro avg      0.886     0.894     0.889       212
weighted avg      0.895     0.892     0.892       212


=== Classification Report: Logistic Regression ===
              precision    recall  f1-score   support

   Fake (0)      0.894     0.874     0.884        87
   Real (1)      0.913     0.928     0.921       125

    accuracy                          0.906       212
   macro avg      0.904     0.901     0.902       212
weighted avg      0.905     0.906     0.905       212


=== Classification Report: LinearSVC ===
              precision    recall  f1-score   support

   Fake (0)      0.908     0.908     0.908        87
   Real (1)      0.936     0.936     0.936       125

    accuracy                          0.925       212
   macro avg      0.922     0.922     0.922       212
weighted avg      0.925     0.925     0.925       212
```

The feature importance of the SVC model shows domains as the primary feature that helped classify data.

	feature	importance
334	real_only_domain	1.643985
176	domain_nytimes	1.334457
302	domain_youtube	1.208168
95	domain_foxnews	1.063416
198	domain_politifact	1.056047
257	domain_time	0.988092
141	domain_medium	0.936203
357	breaking	0.929609
151	domain_nbcnews	0.844883
286	domain_washingtonpost	0.835094
17	domain_archive	0.815029
263	domain_twitter	0.799548
51	domain_cnn	0.791998
193	domain_politico	0.775602
403	florida	0.695292
306	suffix_	0.665216
0	domain_	0.665216
426	just	0.578584
432	letter	0.518025
18	domain_archives	0.516586

Conclusion

Our models achieved the intended goal. With a threshold of 75%, our models moved beyond the 50/50 guessing of binary classification. EDA and feature engineering turned out to the biggest driver of our high accuracy. Extracting domains and then grouping them based on their respective datasets created the strongest indicator of fake news. Not only did this aid in model accuracy, but it provided the cornerstone of our storytelling: if you want to tackle fake news, you must flag the domains that spread fake news.

My approach mirrored other academic studies. For example, a 2024 study by Frontiers in Big Data noted similar concerns of “rapid dissemination” as well as fake-to-real data comparisons [2]. Word clouds visualized news article distributions and identified keywords.

When compared to other studies, I noticed than many of the studies aimed to reduce fake information by prioritizing the content of the article as opposed to focusing on the domain. I was limited by the metadata in my training set, but I was fortunate that the articles themselves were not present. Articles can be thousands of words long, and the processing power for NLPs against this data can be immense. We can achieve a strong level of accuracy purely on the titles and domains. As fake news spreads, the solutions cannot be impractical, such as parsing entire articles that can be generated in minutes. A domain-based approach is more practical as it requires significantly less processing power without dropping accuracy.

Ethical Considerations

The model has shown promise in predicting fake news with high accuracy. A practical use would be to deploy it onto a major social media website such as X or Facebook, where it will detect and block fake news. Nevertheless, there are various ethical concerns involving free speech, spam, robots, misinformation, and business structures.

Classifying information as misinformation usually results in the removal or banning of content. If this happens to impact one political side or candidate more than another, a whole movement might feel they are being attacked. Freedom of speech is a cornerstone of democracy. Combating fake news by deploying models and blocking information must not be weaponized as an attempt to silence people or a movement but rather the unbiased identification of fake news. Individuals should be alerted on the inaccuracies of fake news rather than coldly silencing platforms. It must also be stated that with our model, we primarily targeted domains, not individuals sharing their ideas.

This model is more akin to spam detection. It combines various data points to make a prediction about the validity of the article. While free speech is a tenant of freedom worth respecting and defending, I back reverse searched many of the fake news articles. The domains and the users sharing the fake news often led me to scam websites, often with unsafe unencrypted traffic. The ethical concerns of free speech cannot be tarnished by the evil nature of fake news. These domains are set up to confuse, ensnare, cheat, lie, and polarize readers with lies. Mediums such as YouTube, X, and Facebook must contend with the reality that their multi-trillion-dollar businesses has to treat this fake news as a harm to their users and platform. This likely means banning the bots reposting the fake news and muting the domains, even if it causes users who believe those domains to quit their platforms. If you tell users you are blocking speech, they will feel suspicious, but if they tell them you're blocking spam, they will understand.

We must also consider the users who repost fake news. For many of these individuals, fake news has become an integral part of their identity. They have invested their time (and sometimes financial resources) in believing the falsehoods and bringing them back to the truth is not an effortless endeavor. Imagine reading fake news for a decade only to awaken and discover that your preferred fake news domain has been blocked on Facebook. You would likely experience anger, and that will be the reaction when this model is deployed. However, failing to implement these changes will result in an internet environment where individuals lose their sense of reality as they invest more of themselves into falsehoods.

There is a reason these accounts exist, and data scientists must go beyond blindly classifying fake news. When we research the domains and backgrounds, we find the identifies of fake news peddlers. There are financial incentives with ad traffic on behalf of the platforms and

domains. Foreign states achieve their political and influential goals by brainwashing users. For minimal resources, they can spin up fake news troll farms completed with troll accounts and fake domains. If you ban one, they'll make another. Conspiracy theorists, journalists, and influencers enjoy the polarizing content as it engages their base with more eyeballs for more advertising dollars and influence. And unlike real news which requires research and real events, fake news can be manufactured at the press of a button. Data scientists must consider that companies, foreign states, and thousands of internet personalities share a vested interest in keeping this fake news monster alive and tackling it will result in instability that will be met with outcry.

References:

1. Golovin, Aleksei. "Fake News." *Www.kaggle.com*, 2021, www.kaggle.com/datasets/algord/fake-news.
2. Halyna Padalko, et al. "A Novel Approach to Fake News Classification Using LSTM-Based Deep Learning Models." *Frontiers in Big Data*, vol. 6, no. 1320800, 8 Jan. 2024, https://doi.org/10.3389/fdata.2023.1320800.
3. Norman, Elsa. "Detecting Fake News Using Machine Learning." *Journal of Student Research*, vol. 12, no. 1, 28 Feb. 2023, https://doi.org/10.47611/jsrhs.v12i1.3940. Accessed 5 May 2023.